ContextDrive: A Smartphone Based Context-Aware Driver Assistance System

A Project Report

submitted to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the degree of

Bachelor of Technology

by

Angith Vijayan NE (VML23CS046)

Christo Santhosh (VML23CS082)

Sebastian Mathew Parayidathil (LVML23CS250)

Umarsinan N (VML23CS234)

under the supervision of

Ms. Anju Sreedharan

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VIMAL JYOTHI ENGINEERING COLLEGE CHEMPERI

CHEMPERI P.O., 670632, KANNUR, KERALA, INDIA

October 2026

ContextDrive_Report/certificate/vimal jyothi ima

DEPT. OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the report entitled **ContextDrive: A Smartphone Based Context-Aware Driver Assistance System** submitted by **Angith Vijayan NE** (VML23CS046), **Christo Santhosh** (VML23CS082), **Sebastian Mathew Parayidathil** (LVML23CS250) & **Umarsinan N** (VML23CS234) to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

PROJECT COORDINATORS

Ms. Dinsha P K
Assistant Professor
Computer Science & Engineering
Vimal Jyothi Engineering College

Dr. Nithya V P
Associate Professor
Computer Science & Engineering
Vimal Jyothi Engineering College

Dr. Renji P Cheriyan
Professor
Computer Science & Engineering
Vimal Jyothi Engineering College

PROJECT GUIDE

Ms. Anju Sreedharan
Assistant Professor
Computer Science & Engineering
Vimal Jyothi Engineering College

Place : Chemperi

Head of the Department

Date : 05-10-2026

(Office Seal)

DECLARATION

We hereby declare that the project report **ContextDrive: A Smartphone Based Context-Aware Driver Assistance System**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bona fide work done by us under the supervision of **Ms. Anju Sreedharan**.

This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources.

We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma or similar title of any other University.

Angith Vijayan NE
Christo Santhosh
Sebastian Mathew Parayidathil
Umarsinan N

CHEMPERI
05-10-2026

ACKNOWLEDGEMENT

We are extremely grateful to **Dr. Benny Joseph**, Principal, and **Ms. Divya B**, Head of the Department of Computer Science and Engineering, Vimal Jyothi Engineering College, Chemperi, for providing the facilities and resources required for the successful completion of our project.

We are highly indebted to our project guide, **Ms. Anju Sreedharan**, Assistant Professor, Department of Computer Science and Engineering, for her valuable guidance, constant supervision and encouragement throughout the work. Her suggestions helped us to shape the idea of a smartphone-based context-aware driver assistance system into a well defined project.

We express our sincere thanks to our project coordinators, **Ms. Dinsha P K**, **Dr. Nithya V P** and **Dr. Renji P Cheriyan**, and to all the faculty members of the department for the support and co-ordination extended to us in bringing out this project in time. Our thanks and appreciation also go to the authors of the research papers and other literature that we have referred to in this project.

Last but not the least, we would like to express our gratitude to our parents and friends for their kind cooperation and encouragement.

Angith Vijayan NE
Christo Santhosh
Sebastian Mathew Parayidathil
Umarsinan N

Contents

List of Figures vi

List of Tables vii

List of Abbreviations viii

1 Introduction 1

1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 2
1.4 Scope . . . 2
1.5 Report Organization . . . 3

2 Literature Survey 4

2.1 Existing Systems . . . 4
2.2 Related Work . . . 5
2.2.1 Personalised Driver Risk Assessment with Adaptive Feedback using Crowdsensed Telemetric Data . . . 5
2.2.2 A New CNN Based Object Detection System for Autonomous Mobile Robots Based on Real World Vehicle Datasets . . . 6
2.2.3 Using Contextual Data to Predict Risky Driving Events: A Novel Methodology from Explainable Artificial Intelligence . . . 8
2.2.4 Optimizing Monocular Driving Assistance for Real Time Processing on Jetson AGX Xavier . . . 10
2.2.5 Balancing Transparency and Control: The Impact of AI Explanation Detail on User Perception in Automated Vehicles . . . 11
2.3 Research Gaps . . . 13

3 System Analysis 15

3.1 Proposed System . . . 15
3.2 Feasibility Study . . . 16

3.2.1 Technical Feasibility 16
3.2.2 Operational Feasibility 16
3.2.3 Economic Feasibility 16
3.3 Functional and Non-functional Requirements 17
3.3.1 Functional Requirements 17
3.3.2 Non-functional Requirements 18
3.4 Hardware and Software Requirements 19
3.4.1 Hardware Requirements 19
3.4.2 Software Requirements 19

4 System Design **21**
4.1 System Architecture 21
4.2 UML Diagrams 23
4.2.1 Use Case Diagram 23
4.3 Data Flow Diagrams 24
4.3.1 Level 0 DFD 24
4.3.2 Level 1 DFD 25
4.3.3 Level 2 DFD 25
4.4 ER Diagram and Database Design 26

5 Implementation **28**
5.1 Tools and Technologies 28
5.2 Modules 29
5.3 Algorithms 30
5.3.1 Methods and Techniques 30
5.3.2 Temporal Object Tracking and Approach Detection 31
5.3.3 Context-Aware Risk Assessment 32
5.3.4 Alert Confirmation and Voice Prioritisation 33
5.4 Key Code Snippets 34

6 Testing **36**
6.1 Test Strategy 36
6.2 Test Cases 36
6.3 Results of Unit, Integration, and System Testing 38
6.3.1 Unit Testing Outcomes 38
6.3.2 Integration Testing Outcomes 38
6.3.3 System Testing Outcomes 39

7 Results and Discussion **40**

7.1 Screenshots 40

7.2 Performance Analysis 41

7.3 Comparison with Existing Systems 43

7.4 Discussion 43

8 Conclusion and Future Scope **45**

8.1 Summary 45

8.2 Limitations 45

8.3 Future Enhancements 46

References **47**

List of Figures

4.1 Architecture Diagram of ContextDrive 22
4.2 Use Case Diagram 24
4.3 Data Flow Diagram - Level 0 24
4.4 Data Flow Diagram - Level 1 25
4.5 Data Flow Diagram - Level 2 26

7.1 ContextDrive - Low risk event 41
7.2 ContextDrive - High risk event 41
7.3 ContextDrive - High risk event 41

List of Tables

4.1 Entities of the Logical Data Model 27

5.1 Technologies Used in the ContextDrive Modules 29

7.1 Summary of the Evaluation Results 42
7.2 Comparison of ContextDrive with Existing Systems 43
7.3 Objectives and the Evidence Obtained 44

List of Abbreviations

ADAS	Advanced Driver Assistance Systems
ADB	Android Debug Bridge
AI	Artificial Intelligence
API	Application Programming Interface
CARLA	Car Learning to Act
CNN	Convolutional Neural Network
DFD	Data Flow Diagram
ER	Entity Relationship
GPS	Global Positioning System
HCI	Human-Computer Interaction
IMU	Inertial Measurement Unit
ITS	Intelligent Transport Systems
JSON	JavaScript Object Notation
LBS	Location-Based Services
ML	Machine Learning
OBD-II	On-Board Diagnostics II
SHAP	SHapley Additive exPlanations
UI	User Interface
XAI	Explainable Artificial Intelligence
YOLO	You Only Look Once

Chapter 1

Introduction

1.1 Background

Modern driving can benefit from technologies that provide accurate and timely information about road conditions. Smartphone sensors allow vehicle telemetry and visual data to be collected directly from the dashboard. In this system, the smartphone camera and GPS receiver gather live readings and make the information available to the application.

Machine learning can be used to analyze visual and environmental parameters and identify potential hazards on the road. This helps reduce dependence on manual risk estimation by the driver. Explicit reasoning further improves the system by showing the important factors that influence a safety recommendation, ensuring the driver understands why a warning was issued.

Weather conditions also have an important role in driving safety. Visibility, rainfall, and time of day can severely affect braking distance and reaction times. Therefore, the system combines visual, telemetry, and weather information to provide more useful and context-aware driving guidance.

Contextual interactions are also considered because driving risk depends not only on a single hazard but also on how multiple factors combine. The risk engine provides a comprehensive assessment by evaluating speed, proximity, and environment together. Thus, the system brings different driving information sources together in one cohesive platform.

1.2 Problem Statement

Drivers need to make important split-second decisions regarding speed, following distance, and maneuvering. However, the required information is often available through different and disconnected sources or is difficult to perceive manually. Drivers may have to separately judge

traffic distance, monitor their speed, and mentally adjust for weather or visibility conditions.

Many existing driver assistance systems mainly focus on detecting a specific hazard in isolation, such as a forward collision or lane departure. Such systems may not provide integrated contextual monitoring, transparent explanations for their warnings, or live environmental adjustments in the same affordable platform. Access to reliable, multi-factor safety information can also be difficult for users who do not own vehicles with expensive, built-in advanced driver assistance systems (ADAS).

1.3 Objectives

- To develop a smartphone based context-aware driver assistance system for real time driving risk assessment.
- To integrate camera, GPS, weather, and time based information for comprehensive driving context analysis.
- To generate a risk score and provide timely, transparent safety recommendations.
- To improve driver awareness and promote safer driving without requiring additional hardware.

1.4 Scope

The ContextDrive system covers several major areas of driver assistance and safety monitoring. It provides real-time object detection using a smartphone camera, GPS-based telemetry insights, weather-aware risk adjustments, and time-based contextual analysis. The system also includes explicit reasoning logic for understanding risk predictions and a multi-dimensional context vector for analyzing combined driving factors. A responsive dashboard interface allows drivers to access these features directly from a mounted mobile device.

The system also provides on-device data processing for ensuring privacy and low-latency performance without requiring specialized external hardware. It is intended to support drivers in maintaining situational awareness and making safer decisions rather than replacing their attention or active vehicle control.

1.5 Report Organization

The remainder of this report is organized as follows. Chapter 2 surveys existing driver assistance systems and related research, and identifies the gaps that ContextDrive addresses. Chapter 3 presents the system analysis, including the proposed system, feasibility study, functional and non-functional requirements, and the hardware and software requirements. Chapter 4 describes the system design through the architecture, UML diagrams, data flow diagrams, and the data model. Chapter 5 explains the implementation, covering the tools and technologies, modules, algorithms, and key code snippets. Chapter 6 reports the test strategy, test cases, and the results of unit, integration, and system testing. Chapter 7 presents the results and discussion, including application screenshots, performance analysis, and a comparison with existing systems. Chapter 8 concludes the report with a summary, the limitations of the work, and future enhancements.

Chapter 2

Literature Survey

This chapter surveys the existing approaches to driver assistance and driving risk assessment, reviews the research papers that informed the design of ContextDrive, and summarizes the gaps in the current work that the proposed system sets out to address.

2.1 Existing Systems

Existing solutions for improving driving safety fall broadly into the following categories.

- **Factory-fitted ADAS:** Many existing driver assistance systems focus on detecting a specific hazard in isolation, such as a forward collision or lane departure. They depend on built-in vehicle hardware, so drivers who do not own vehicles with expensive advanced driver assistance systems cannot benefit from them.
- **Telematics-based driver monitoring:** Traditional approaches rely on proprietary in-vehicle hardware such as On-Board Diagnostics (OBD-II) dongles or dedicated inertial measurement units (IMUs). More recent work uses the GPS traces of ordinary smartphones to derive indicators such as harsh braking, rapid acceleration, overspeeding, and aggressive cornering [1]. These systems lack visual awareness of the road scene.
- **Camera-based perception on edge devices:** Lightweight detectors such as SSD with a MobileNetV2 backbone make real-time vehicle detection possible on constrained hardware [2]. Monocular camera systems offer an affordable alternative to LiDAR and radar, but the reported optimizations target a specific embedded platform, the NVIDIA Jetson AGX Xavier [3].
- **Explainable context-aware risk models:** Machine learning models trained on large telematics datasets, combined with explainable AI techniques such as SHAP, show how

speed, weather, and traffic context jointly create risk [4]. Such analysis is suited to post-trip evaluation on backend servers rather than to real-time use on a phone.

2.2 Related Work

The following papers were reviewed in detail. For each paper, the methodology, advantages, and disadvantages are summarized.

2.2.1 Personalised Driver Risk Assessment with Adaptive Feedback using Crowdsensed Telemetric Data

The rapid evolution of Intelligent Transport Systems (ITS) has brought significant advancements in how driver behavior is monitored and evaluated. Traditional approaches often relied on expensive, proprietary in-vehicle hardware such as On-Board Diagnostics (OBD-II) dongles or dedicated inertial measurement units (IMUs). However, recent trends have shifted towards utilizing the ubiquitous presence of smartphones. This paper by Auwal Sagir Muhammad et al., published in IET Intelligent Transport Systems (IET, 2025), presents a comprehensive framework for assessing driver risk using such crowdsensed telemetric data. The study focuses on transforming raw GPS traces and temporal data into structured, actionable driving risk indicators. By avoiding complex hardware, this approach democratizes driver profiling, making it accessible to a wider population and enabling large-scale, crowdsourced data collection for traffic safety analysis.

2.2.1.1 Methodology

The authors utilized a vast dataset of crowdsensed telemetric data collected from standard smartphones to engineer several highly specific GPS-derived driver risk indicators. The core of their methodology involved developing algorithms to filter out GPS noise and smooth the trajectory data before analysis. From this refined data, they established specific numerical thresholds for critical driving events. For instance, harsh braking was calculated by analyzing negative acceleration thresholds over consecutive GPS points, while rapid acceleration was identified through sudden positive velocity spikes. Overspeeding was determined by continuously comparing the GPS-derived vehicle speed against mapped speed limits obtained from external routing APIs. Furthermore, the turning rate was computed by analyzing the delta in GPS bearing over time, identifying aggressive cornering.

Crucially, the system incorporated contextual flags based on the driving environment. Nighttime driving and rainy weather conditions were used to dynamically modulate the risk

thresholds, acknowledging that a safe speed in broad daylight might be inherently dangerous during a nighttime downpour. These engineered indicators provided a structured, rule-based approach for extracting interpretable driving risk features from raw telemetric data. Various machine learning models, including Support Vector Machines and Random Forests, were then applied to evaluate how these engineered indicators effectively capture driving volatility and predict overall driver risk profiles.

2.2.1.2 Advantages

The proposed system introduces validated feature engineering methods for transforming raw GPS, temporal, and weather information into highly meaningful driver behavior indicators. A significant advantage of this system is its inherent scalability and cost-effectiveness; by relying entirely on crowdsensed smartphone data, it eliminates the need for any specialized vehicle modifications. Furthermore, by relying on well-defined, threshold-based rules rather than opaque deep learning models for feature extraction, the approach provides an easily interpretable solution. This transparency allows the system to generate adaptive, understandable feedback for the driver, explaining exactly which behaviors (e.g., frequent harsh braking) contributed to their risk profile.

2.2.1.3 Disadvantages

Despite its strengths, the methodology presents notable limitations. One major limitation is that the proposed thresholds for harsh braking, acceleration, and turning were validated within a specific machine learning pipeline and a particular regional context. Driving cultures and road topologies vary drastically across the globe; therefore, these rigid numerical values may require significant recalibration when applied to different geographical driving environments. Additionally, a pure telematics approach lacks visual context. If the system detects harsh braking, it cannot determine whether the driver was driving aggressively or reacting appropriately to a child suddenly running into the street. This lack of visual situational awareness highlights the need for systems that integrate telematics with camera-based perception, a gap that the ContextDrive system aims to bridge.

2.2.2 A New CNN Based Object Detection System for Autonomous Mobile Robots Based on Real World Vehicle Datasets

The deployment of accurate and reliable object detection models is a cornerstone of modern autonomous navigation and advanced driver-assistance systems. While cloud-based deep learning models have achieved remarkable accuracy, they suffer from latency and require continuous internet connectivity, making them unsuitable for critical driving safety applications. This paper by Udink Aulia et al., published in Heliyon (Elsevier, 2024), addresses

this challenge by presenting a lightweight object detection system specifically designed for autonomous mobile robots and edge devices. The study focuses on balancing real-time processing speeds with high detection accuracy in real-world environments, heavily utilizing transfer learning to adapt generic models to specific navigational tasks.

2.2.2.1 Methodology

To achieve real-time performance on hardware-constrained edge devices, the authors developed a vehicle detection system utilizing the SSD (Single Shot MultiBox Detector) framework paired with a MobileNetV2 backbone and an FPN-Lite (Feature Pyramid Network) architecture. The MobileNetV2 backbone employs depthwise separable convolutions to drastically reduce the computational complexity and parameter count compared to standard convolutional neural networks (CNNs). The FPN-Lite structure allows the model to construct high-level semantic feature maps at all scales, significantly improving the detection of small or distant vehicles, which is critical for early collision warning.

To overcome the limitations of generic pre-trained models, which often fail to generalize to specific deployment environments, the authors employed extensive transfer learning. The pre-trained detector was fine-tuned on a newly collected and meticulously annotated real-world vehicle dataset. This dataset was specifically curated to represent the lighting, weather, and traffic conditions the autonomous robots would actually face. Several training configurations, including various learning rates, batch sizes, and data augmentation techniques (such as random cropping, flipping, and color jittering), were tested. The models were then rigorously benchmarked to identify the optimal hyperparameters that maximize the mean Average Precision (mAP) while keeping inference latency low enough for real-time mobile robot navigation.

2.2.2.2 Advantages

The study provides compelling empirical evidence that transfer learning and fine-tuning of a lightweight SSD MobileNetV2 detector can significantly improve vehicle detection accuracy while maintaining strict real-time performance. This provides a highly practical, ready-to-deploy approach for domain-specific implementation on resource-constrained edge devices. By demonstrating that high accuracy does not necessitate massive computing power, the research paves the way for advanced perception systems to be deployed on affordable hardware, making safety technologies more accessible.

2.2.2.3 Disadvantages

The primary disadvantage of this approach is the phenomenon known as “domain shift.” The detection performance of the fine tuned model heavily depends on the quality, size, and diversity of the fine tuning dataset. If the autonomous robot or vehicle encounters conditions

not well represented in the training data, such as sudden heavy fog, snow, or nighttime driving in poorly lit areas, the model's accuracy degrades rapidly. Consequently, adapting the model to completely new environments requires substantial, time consuming additional data collection and manual annotation efforts. This highlights the limitation of relying solely on a fixed visual dataset without integrating dynamic environmental context APIs to adjust system sensitivity.

2.2.3 Using Contextual Data to Predict Risky Driving Events: A Novel Methodology from Explainable Artificial Intelligence

Traditionally, driving risk assessment has focused heavily on isolated telematics variables, such as absolute speed or the frequency of hard braking. However, driving is an inherently contextual activity; a speed of 60 km/h might be perfectly safe on a dry highway but extremely dangerous on an icy residential street. This paper by Leandro Masello et al., published in Accident Analysis & Prevention (Elsevier, 2023), investigates how these surrounding contextual factors fundamentally influence risky driving events. The research introduces a novel methodology based on explainable artificial intelligence (XAI) to move beyond isolated risk variables, analyzing how combined environmental and traffic contexts compound to create hazardous driving situations. By bridging the gap between raw data collection and human-understandable reasoning, this study highlights the growing need for transparency in modern intelligent transport systems.

2.2.3.1 Methodology

The researchers utilized large-scale, naturalistic telematics data collected from fleets of vehicles over an extended period. This comprehensive dataset included standard variables like GPS coordinates, vehicle speed, and acceleration profiles. Crucially, they augmented this telematics data with a rich array of contextual driving factors obtained from external geographical and meteorological databases. These included varying speed limits, real-time weather conditions (precipitation levels, visibility limits), traffic density metrics, road characteristics (such as urban vs. rural settings, intersection proximity, and road curvature), and temporal variables (time of day, day of week).

They trained sophisticated supervised machine learning models, specifically tree-based ensembles like Random Forest and Gradient Boosting Machines (GBM), to predict the occurrence of risky driving events based on this multi-dimensional data. Tree-based models were chosen for their robust ability to handle non-linear relationships and heterogeneous data types without extensive preprocessing. However, to interpret the complex, non-linear interactions between these variables learned by the black-box models, they applied advanced XAI techniques, specifically SHAP (SHapley Additive exPlanations). Based on cooperative game theory, SHAP values allowed the researchers to quantify the exact marginal contribution

of each individual contextual factor toward the likelihood of a risky driving event occurring. For instance, the SHAP analysis could reveal precisely how much "heavy rain" contributed to a model predicting a "high risk" event compared to "speeding." This provided a highly granular, feature-by-feature view of how risk is dynamically formed in real-time.

2.2.3.2 Advantages

The methodology effectively and quantitatively demonstrates that driving risk is determined by the combined, compounded influence of multiple contextual factors rather than isolated telematics variables. It provides a strong, evidence-based foundation for context-aware risk assessment. For example, the study proved that the danger of a sharp turn is exponentially increased when combined with poor visibility, a correlation that simple rule-based telematics systems often miss.

Furthermore, the use of SHAP values transforms opaque machine learning predictions into transparent insights. This transparency is vital for the widespread adoption and psychological acceptance of driver assistance systems. By showing a driver exactly *why* a situation was classified as risky (e.g., “High risk due to combination of moderate speed and heavy rain”), the system builds user trust and encourages active behavioral correction much more effectively than a generic, unexplained warning beep. It shifts the paradigm from a reactive warning system to a proactive, educational driving companion.

2.2.3.3 Disadvantages

Despite its significant contributions to explainability, the proposed methodology presents practical challenges for real-world deployment on edge devices. The system relies heavily on large-scale historical telematics data for supervised machine learning training. As a result, the models require extensive, computationally expensive retraining and recalibration when applied to completely different driving environments, varying vehicle types, or new geographical datasets where the baseline contextual risk factors might fundamentally differ.

Additionally, calculating SHAP values for complex ensemble models is incredibly resource-intensive. While this methodology is highly suitable for post-trip analysis, driver debriefing, or batch processing on powerful backend servers, running these XAI algorithms continuously in real-time on a resource-constrained edge device or consumer smartphone presents a massive computational bottleneck. For real-time applications like ContextDrive, simpler, deterministic rule-based engines or highly optimized, quantized neural networks must be employed to provide instant feedback without draining the device’s battery or causing thermal throttling, even if they sacrifice some of the deep nuance provided by full SHAP analysis.

2.2.4 Optimizing Monocular Driving Assistance for Real Time Processing on Jetson AGX Xavier

While autonomous vehicles often rely on expensive, multi-modal sensor suites including LiDAR, radar, and stereo cameras, consumer-grade Advanced Driver Assistance Systems (ADAS) must balance performance with cost and portability. Monocular camera systems offer a highly affordable alternative but require sophisticated computer vision algorithms to extract depth and spatial awareness from a single lens. This paper by Huy-Hung Nguyen et al., published in IEEE Access (IEEE, 2024), presents a monocular camera-based ADAS optimized specifically for high-performance edge computing, aiming to perform complex road scene understanding in real time without specialized depth sensors.

2.2.4.1 Methodology

The authors developed a unified vision pipeline designed to simultaneously handle multiple perception tasks, such as vehicle detection, pedestrian tracking, and traffic sign recognition, using a single live monocular camera feed. The core of their methodology involved deeply optimizing these lightweight object detection models for deployment on the NVIDIA Jetson AGX Xavier, a powerful embedded edge computing platform.

To achieve high frame rates, they integrated aggressive optimization techniques. This included tensor optimization using NVIDIA's TensorRT, which fuses network layers and optimizes memory access patterns. Furthermore, they employed reduced precision inference, converting the neural network weights from standard 32-bit floating-point (FP32) to 16-bit floating-point (FP16) or 8-bit integer (INT8) representations. This quantization drastically reduces memory bandwidth requirements and accelerates mathematical operations. The performance of these highly optimized models was then rigorously benchmarked against standard baseline architectures to meticulously evaluate the trade-offs between frame rate throughput, processing latency, and overall detection accuracy in highly dynamic driving scenarios.

2.2.4.2 Advantages

The research convincingly demonstrates that a single monocular camera, when paired with heavily optimized lightweight object detection algorithms, can reliably perform complex, real-time perception tasks required for driver assistance. By achieving high frame rates and low latency, the system validates the feasibility of comprehensive camera-based road scene understanding without needing specialized, expensive sensors like LiDAR. This approach significantly reduces the hardware cost and complexity of implementing ADAS features in standard vehicles.

2.2.4.3 Disadvantages

The primary limitation of this research lies in its hardware specificity. The system's architecture, memory management, and specific tensor optimization enhancements are heavily tailored specifically for the NVIDIA Jetson AGX Xavier hardware platform and its unique GPU architecture. While powerful, the Jetson AGX Xavier is a relatively expensive embedded system not typically found in consumer smartphones. Consequently, the deployment strategy and acceleration techniques developed in this study are not directly transferable to standard smartphone-based implementations, which rely on different neural processing units (NPUs) and frameworks like Google LiteRT (TensorFlow Lite), requiring entirely different optimization paradigms.

2.2.5 Balancing Transparency and Control: The Impact of AI Explanation Detail on User Perception in Automated Vehicles

As artificial intelligence takes a profoundly larger role in vehicle control and driver assistance, the human-computer interaction (HCI) aspect of these systems has emerged as a critical bottleneck for widespread public adoption. A significant, persistent barrier to the acceptance of AI-driven safety systems is the well-documented "black box" problem, where users do not understand the underlying mechanisms dictating why a system made a certain sudden decision. This opacity often leads to "automation surprise," resulting in either a complete lack of trust and system disabling, or conversely, dangerous over-reliance. This paper by Jakob Peintner et al., published in Transportation Research Interdisciplinary Perspectives (Elsevier, 2025), rigorously investigates the psychological and behavioral effects of AI explainability on user trust and perception. The core objective of the study explores how varying levels of detail and complexity in safety explanations affect a user's cognitive experience, aiming to pinpoint the optimal balance between providing necessary transparency to build trust and preventing cognitive overload that could impair situational awareness.

2.2.5.1 Methodology

To safely, ethically, and consistently evaluate user reactions to critical, potentially dangerous driving events, the authors designed and conducted a comprehensive virtual reality (VR) driving study. Human participants were placed in a highly realistic, immersive simulated Level-4 highly automated vehicle environment. In this setup, the vehicle handles the vast majority of the dynamic driving tasks, placing the human participant primarily in a supervisory or passenger role. Throughout the controlled simulation, the vehicle deliberately encountered various complex, randomized traffic scenarios requiring sudden automated interventions, such as a pedestrian unexpectedly darting into the road or a lead vehicle braking abruptly.

To isolate the effect of explainability, the researchers systematically tested four distinct

AI explanation levels presented via an in-cabin graphical interface during these critical interventions: “None” (the vehicle acts completely autonomously without providing any rationale), “How” (the system simply describes the physical action it is taking, e.g., “Braking hard”), “How & Why” (the system describes both the action and its underlying immediate reasoning, e.g., “Braking hard because of a pedestrian ahead”), and finally “Contrastive & Counterfactual” (the system exhaustively explains why an alternative action was inherently worse and not taken, e.g., “Braking hard because swerving into the left lane would cause a catastrophic collision with oncoming traffic”).

To capture a holistic view of the user experience, the researchers employed a multi-modal data collection approach. Beyond standard detailed post-scenario psychometric surveys (utilizing validated Trust in Automation scales), they continuously monitored physiological and behavioral metrics during the VR simulation. This included advanced eye-tracking to measure pupil dilation as a proxy for cognitive load and stress levels, allowing them to quantitatively evaluate the psychological toll of processing each explanation level during an emergency.

2.2.5.2 Advantages

The study provides robust, empirical evidence addressing a core HCI challenge in vehicle automation, shifting the discourse from anecdotal UI design to data-driven psychology. The research demonstrates clearly and quantitatively that concise “How & Why” explanations offer the absolute most effective balance between system transparency and user usability. The physiological data proved that while providing no explanation severely degrades trust and spikes anxiety (due to automation surprise), providing overly complex, paragraph-style contrastive explanations drastically increases cognitive load without yielding any significant improvement in perceived safety or trust.

This research establishes a clear, evidence-based psychological framework for designing understandable and trustworthy UI/UX for driver assistance recommendations. It validates the design philosophy that ADAS interfaces must immediately answer the user’s instinctive “Why is the car doing this?” question concisely. This perfectly aligns with modern, context-aware systems like ContextDrive, which strive to distill complex risk calculations into immediate, digestible visual feedback (such as color-coded risk rings paired with short, specific “Why” and “Advice” strings) to build rapid user consensus.

2.2.5.3 Disadvantages

Despite the rigorous experimental design, the primary limitation of this study is its ecological validity regarding active driver assistance. The findings are strictly based on passenger interactions within a simulated Level-4 highly automated driving environment, where the user is essentially a passive observer with the luxury of time to process graphical notifications.

Consequently, these results may not directly, safely generalize to real-time, manual driver assistance systems (Level 1/2) where the human user is actively controlling the vehicle's steering and pedals, and requires split-second, visceral alerts to avoid collisions. In highly dynamic, high-stakes manual driving situations, even reading a relatively concise textual "How & Why" explanation on a smartphone screen might cause a critical visual distraction, drawing the driver's eyes away from the road precisely when their attention is needed most. For active real-time ADAS, explanations must be delivered through highly intuitive, non-intrusive modalities such as specific, distinct audio tones, haptic feedback, or peripheral color changes rather than relying primarily on explicit textual descriptions, a crucial real-world nuance that this passenger-centric VR study does not fully encompass.

2.3 Research Gaps

The review of the above work reveals the following gaps.

1. **No visual context in telemetry-only approaches.** A purely GPS-based system cannot tell whether harsh braking was caused by aggressive driving or by a child running into the street [1].

2. **Fixed visual models ignore environmental context.** The accuracy of a fine-tuned detector degrades under conditions not represented in its training data, such as fog or night driving, and the work does not adjust sensitivity using dynamic environmental information [2].

3. **Explainability is too expensive for real-time edge use.** Computing SHAP values for ensemble models is resource-intensive, so such analysis suits post-trip processing, whereas real-time use on a smartphone requires simpler, deterministic rule-based engines [4].

4. **Hardware-specific optimization.** The acceleration techniques developed for an embedded GPU platform do not transfer directly to smartphones, which rely on different processors and frameworks such as Google LiteRT [3].

5. **Explanations for manual driving.** Concise "How & Why" explanations were validated only with passengers in a simulated Level-4 vehicle. For active manual driving, explanations should be short and delivered through non-intrusive modalities rather than long text [5].

ContextDrive addresses these gaps by combining a smartphone camera, GPS telemetry, weather, and time of day in a single on-device pipeline. It uses a lightweight YOLOv8 Nano

detector running on Google LiteRT, and a deterministic rule-based risk engine that adjusts its sensitivity using the environmental context. Each warning is accompanied by a short “Why” and “Advice” explanation, in line with the findings on concise explanations.

Chapter 3

System Analysis

3.1 Proposed System

The proposed system is a smartphone-based platform that combines computer vision, GPS telemetry, and weather intelligence to give drivers reliable and context-aware safety assistance. The system is divided into five modules.

1. **Vision and Object Detection Module:** Captures real time road scenes using the smartphone camera and detects vehicles and obstacles using a lightweight YOLOv8 Nano model.
2. **Telemetry and GPS Module:** Collects real-time speed, location, and bearing data from the smartphone's built-in GPS receiver to track driving behavior.
3. **Environmental Context Module:** Retrieves live weather and visibility conditions via external APIs based on the current location.
4. **Risk Engine and Reasoning Module:** Combines visual, telemetry, and environmental data to calculate a continuous driving risk score and provides transparent explanations for its assessment.
5. **User Interface Module:** Displays the live camera feed, current risk status, warnings, and safety recommendations through an intuitive dashboard.

The working of the system in brief is as follows. The driver mounts the smartphone on the dashboard and starts the application. The camera begins capturing the road ahead, while the GPS receiver continuously tracks vehicle speed and location. The environmental module simultaneously checks the local weather conditions. The risk engine processes these inputs together evaluating the distance to detected vehicles, current speed, and weather-related visibility to determine a driving risk score. If a hazard or unsafe condition is detected, the reasoning module generates a transparent explanation. Finally, the user interface displays the

live feed alongside context-aware visual warnings and safety recommendations to assist the driver.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The system uses computer vision algorithms, smartphone sensors, weather APIs, and mobile application technologies, all of which are mature and well documented. It can be developed using open source frameworks like Flutter and Google LiteRT, alongside lightweight object detection models such as YOLOv8 Nano. Since modern smartphones are equipped with high quality cameras and GPS receivers, the integration of on-device processing without external computing hardware is technically possible. Existing research has already shown that monocular camera based perception and telematics based risk assessment work in practice for driver assistance.

3.2.2 Operational Feasibility

The system provides a simple and intuitive dashboard interface for drivers with different technical backgrounds. It delivers real-time risk assessment, visual warnings, and transparent safety recommendations to improve driving decisions. Since the application runs entirely on the driver's smartphone mounted on the dashboard, it requires minimal setup and integrates easily into daily driving routines. The clear, explicit reasoning provided alongside alerts ensures that the system is easily understandable and builds user trust without causing cognitive overload.

3.2.3 Economic Feasibility

The project reduces the likelihood of accidents through proactive risk assessment and timely safety warnings. The development cost is low because open source software frameworks and freely available AI models are used. Unlike traditional Advanced Driver Assistance Systems (ADAS) that require expensive, specialized vehicle hardware, this system leverages the smartphone the user already owns. The minimal cost associated with standard mobile data usage for weather APIs makes the system highly affordable. The expected improvement in driver safety and accident prevention is significantly greater than the minimal operational cost of the system.

3.3 Functional and Non-functional Requirements

3.3.1 Functional Requirements

3.3.1.1 Object Detection

The system shall detect vehicles and obstacles in real time using the smartphone's rear camera and a lightweight object detection model.

3.3.1.2 Contextual Risk Assessment

The system shall continuously calculate a driving risk score based on combined inputs such as vehicle speed, proximity to detected objects, and environmental conditions.

3.3.1.3 Transparent Safety Recommendations

The system shall provide clear, explicit reasoning for its risk assessments. The explanations shall help users understand the important factors influencing the current safety recommendations.

3.3.1.4 Telemetry and GPS Analysis

The system shall use the smartphone's GPS receiver to gather live speed, location, and bearing information. This data shall be used to evaluate driving behavior and compute telemetry-based risk indicators.

3.3.1.5 Weather and Environmental Monitoring

The system shall obtain current weather information using the OpenWeatherMap API. Weather information, such as rain and visibility, shall be used to adjust risk calculations dynamically.

3.3.1.6 Real-Time Visual Feedback

The system shall provide an intuitive dashboard interface displaying the live camera feed, current risk status, and safety alerts directly on the mounted mobile device.

3.3.1.7 Alerts and Notifications

The system shall issue timely, context-aware warnings to the driver regarding unsafe following distances, speed violations, and adverse weather conditions.

3.3.2 Non-functional Requirements

The following non-functional requirements describe the qualities that the system must exhibit in addition to its functions.

3.3.2.1 Performance and Responsiveness

The system shall process camera frames and update the risk assessment on the device with low latency, so that warnings reach the driver in time to be useful.

3.3.2.2 Privacy

All video processing shall be performed on the device. Camera frames shall not leave the smartphone, and the only external request shall be the weather query made using the current coordinates.

3.3.2.3 Usability and Low Distraction

The interface shall be simple enough for drivers with different technical backgrounds. Warnings shall be conveyed through color-coded risk indicators, short “Why” and “Advice” messages, and voice alerts, so that the driver does not have to read long text while driving.

3.3.2.4 Reliability and Data Quality

The system shall handle noisy GPS readings, and unavailable or stale inputs, without crashing. Missing or outdated inputs shall be reported as data limitations and shall not be treated as favourable driving conditions.

3.3.2.5 Transparency

Every warning shall be accompanied by an explanation of the factors that led to it, so that the driver can understand why the alert was issued.

3.3.2.6 Portability and Affordability

The application shall run on a standard smartphone using a cross-platform framework, without any specialized external hardware or vehicle modification.

3.3.2.7 Maintainability and Testability

The system shall be organized into independent modules with configurable rules and thresholds, so that individual components can be tested separately and the system can be exercised with simulated inputs.

3.3.2.8 Safety Scope

The system shall assist the driver in maintaining situational awareness. It is not intended to replace the driver's attention or active control of the vehicle.

3.4 Hardware and Software Requirements

3.4.1 Hardware Requirements

3.4.1.1 Smartphone Device

A modern smartphone serves as the primary target platform for the application, providing the necessary computational capability, memory, and internet connectivity to run the system entirely on-device without requiring external processing hardware.

3.4.1.2 Rear Camera

The smartphone's rear camera is used to capture live video feeds of the road scene. The continuous stream of frames is passed to the object detection model to identify vehicles and vulnerable road users in real-time.

3.4.1.3 GPS Receiver

The built-in GPS receiver of the smartphone is utilized to gather real-time telemetry data. It provides continuous updates on vehicle speed, location coordinates, and bearing, which are critical for driving behavior analysis and risk assessment.

3.4.1.4 Internet Connectivity

A stable mobile data or Wi-Fi connection is required to retrieve real-time weather information and forecasts through external APIs, ensuring the risk assessment engine operates with up-to-date environmental context.

3.4.2 Software Requirements

The ContextDrive system utilizes a modern mobile application architecture. It is primarily developed using Dart, Flutter, Google LiteRT, and integrates advanced object detection models and weather services.

3.4.2.1 Dart

Dart is the primary programming language used for the application. It provides a robust, object-oriented framework for writing both the user interface and the underlying business logic, including the risk assessment engine and telemetry processing.

3.4.2.2 Flutter

Flutter is used as the cross-platform application framework to develop the user interface. It provides a modular and responsive foundation for features such as the live camera dashboard, risk status displays, and setting configurations, ensuring consistent performance.

3.4.2.3 Google LiteRT

Google LiteRT (formerly TensorFlow Lite) is employed as the on-device machine learning inference framework. It enables the application to execute complex object detection algorithms directly on the smartphone with low latency and high efficiency.

3.4.2.4 YOLOv8 Nano (YOLOv8n)

YOLOv8 Nano is the lightweight object detection model integrated into the system. It is specifically optimized for edge devices and is responsible for detecting and tracking vehicles and other obstacles in the camera's field of view in real time.

3.4.2.5 OpenWeatherMap API

The OpenWeatherMap API is used to fetch current weather conditions based on the user's GPS coordinates. This environmental data provides critical context, such as rain or poor visibility, which is integrated directly into the contextual risk assessment calculations.

Chapter 4

System Design

The design of the system is user centric and modular. This chapter presents the system architecture, the use case diagram, the data flow diagrams, and the data model of ContextDrive.

4.1 System Architecture

An architecture diagram provides an overall view of the major components of a software system and the connections between them. The architecture of the ContextDrive system consists of five main layers, as shown in Figure 4.1.

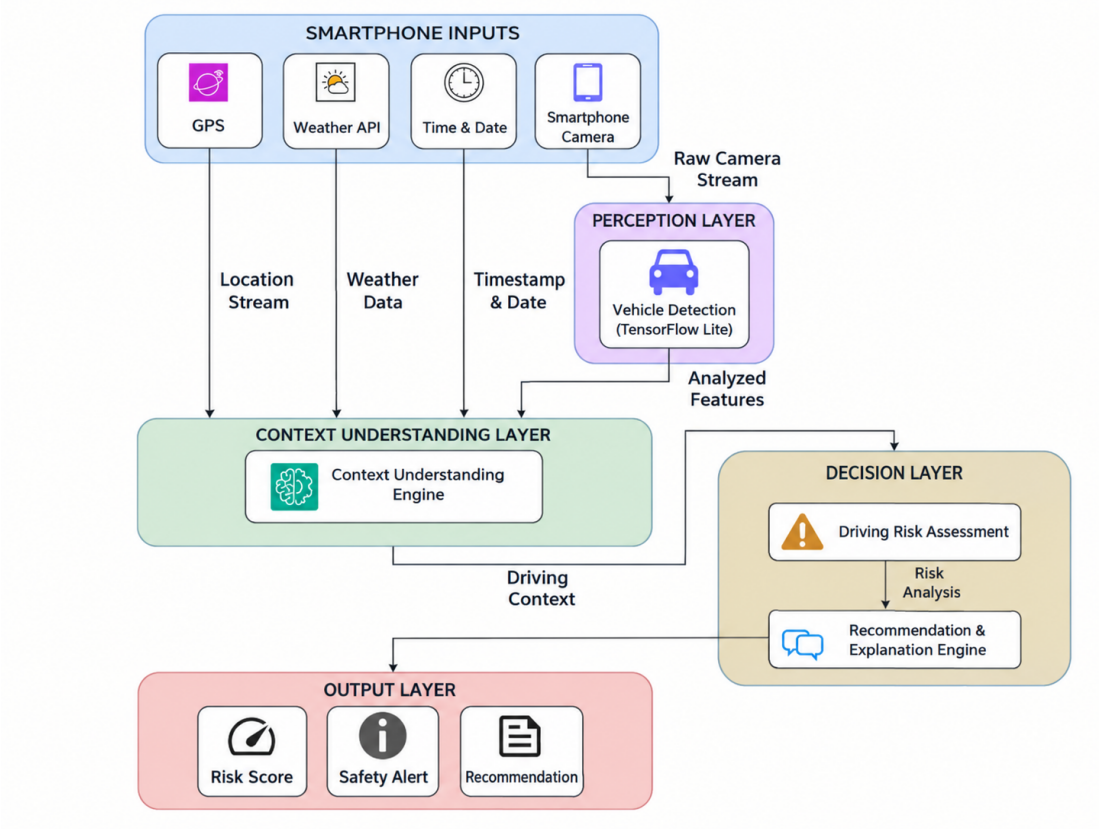


Figure 4.1: Architecture Diagram of ContextDrive

1. **Input Layer:** This layer is the primary source of raw data for the system, originating from a user's smartphone. It groups several distinct data sources:
   - *GPS:* Outputs a location stream to the context understanding layer.
   - *Weather API:* Outputs weather data based on the current location.
   - *Time & Date:* Outputs timestamp and date information.
   - *Smartphone Camera:* Outputs a raw camera stream to the perception layer.
2. **Perception Layer:** This layer processes raw sensor data from the input layer to detect environmental features. The vehicle detection engine analyzes visual data, receiving the raw camera stream and outputting analyzed features regarding detected vehicles and objects.
3. **Context Understanding Layer:** This central layer consolidates diverse data points to create a comprehensive understanding of the current driving environment and situation. The context understanding engine builds a high-level situational model by receiving the location stream, weather data, timestamp, and analyzed features, ultimately generating a critical data stream called the driving context.

4. **Decision Layer:** This layer analyzes the comprehensive driving context to assess risk and formulate advice in two connected stages:
    - *Driving Risk Assessment:* Evaluates the context data for potential hazards and risk levels, generating a risk analysis stream.
    - *Recommendation & Explanation Engine:* Generates actionable advice and the rationale behind that advice based on the risk analysis, resulting in a consolidated assessment and recommendation stream.
5. **Output Layer:** This final layer presents the system's findings and advice to the user. It receives the high-level driving context and the assessment and recommendation data. The presentation modules include the Risk Level, Safety Alert, and Recommendation.

4.2 UML Diagrams

4.2.1 Use Case Diagram

The use case diagram illustrates the main functionalities of the ContextDrive system and the interactions of its primary actor, the driver. The driver can start a trip, view the live camera dashboard, and receive real-time driving risk assessments based on speed and proximity. The driver can also review the transparent safety explanations generated by the system to understand the factors influencing the current warnings.

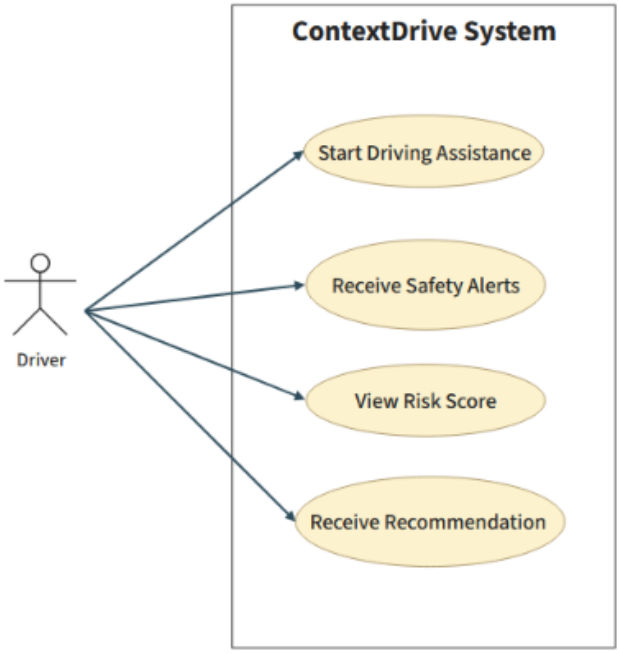


Figure 4.2: Use Case Diagram

4.3 Data Flow Diagrams

4.3.1 Level 0 DFD

Level 0 represents the complete ContextDrive system as a single process, as shown in Figure 4.3. It illustrates the high-level flow of information:

- **Driving Context Data (Input):** Raw environmental and sensor data (location, weather, time, camera) fed into the system.
- **ContextDrive (Processing):** The core engine that receives the raw data, analyzes the driving situation, and evaluates potential risks.
- **Risk Alerts & Driving Recommendations (Output):** The final result generated by ContextDrive, delivering safety warnings and driving advice to the user.



Figure 4.3: Data Flow Diagram - Level 0

4.3.2 Level 1 DFD

Level 1 decomposes the ContextDrive system into major processes, as shown in Figure 4.4. The flow is structured as follows:

- **Inputs:** Four external data sources feed into the central system:
  - *Smartphone Camera:* Sends Camera Frames.
  - *Smartphone GPS:* Sends Speed / Location.
  - *Weather API:* Sends Weather Data.
  - *Smartphone Clock:* Sends Time & Date.
- **ContextDrive System:** The central processing hub that receives and combines all four input streams.
- **Driver:** The final recipient who receives the Risk Score, Safety Alerts, and Recommendations generated by the ContextDrive System.

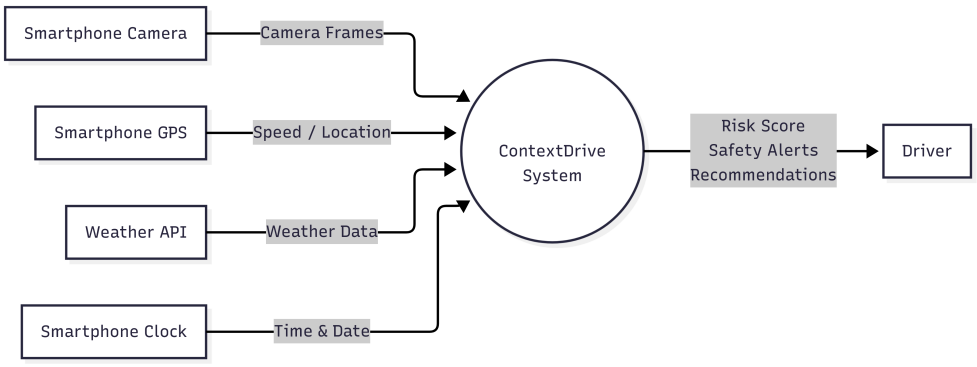


Figure 4.4: Data Flow Diagram - Level 1

4.3.3 Level 2 DFD

Level 2 DFD provides a more detailed view of the internal sub-processes within the core modules of the system, as shown in Figure 4.5.

- **Inputs:**
  - *Smartphone Camera:* Sends Camera Frames to the Vehicle Detection process.
  - *Smartphone GPS:* Sends Speed/Location to the Context Understanding process.
  - *Weather API:* Sends Weather Data to the Context Understanding process.
  - *Smartphone Clock:* Sends Time & Date to the Context Understanding process.

- **Processes:**
  - *Vehicle Detection:* Receives Camera Frames and outputs Vehicle Information to the Context Understanding process.
  - *Context Understanding:* Combines Vehicle Information, Speed/Location, Weather Data, and Time & Date to output Driving Context to the Driving Risk Assessment process.
  - *Driving Risk Assessment:* Analyzes the Driving Context and sends a Risk Score to the Recommendation & Explanation process.
  - *Recommendation & Explanation:* Takes the Risk Score and sends the final Alert and Recommendation to the Driver.
- **Output:**
  - *Driver:* The final recipient who receives the Alert and Recommendation from the Recommendation & Explanation process.

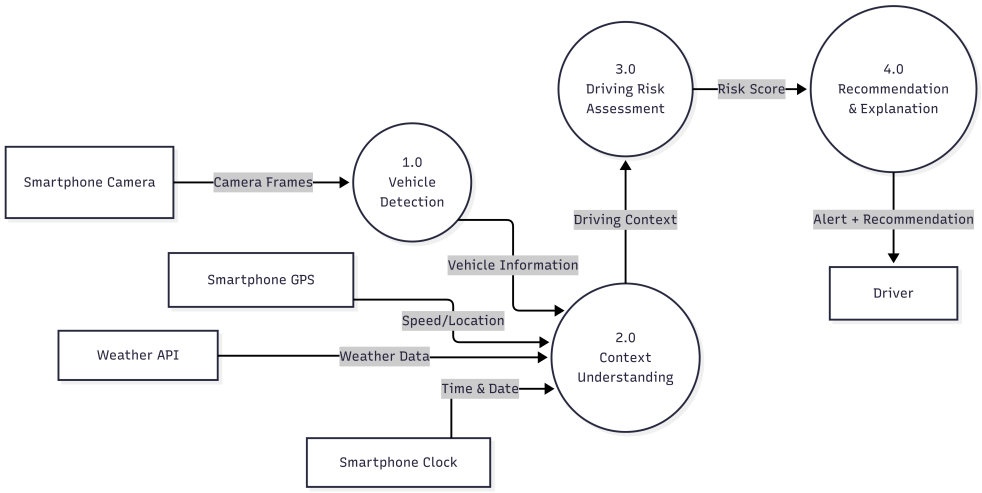


Figure 4.5: Data Flow Diagram - Level 2

4.4 ER Diagram and Database Design

ContextDrive performs its processing entirely on the device and does not depend on a central server database. The information handled during a trip can nevertheless be described by a logical data model. A `Trip` groups the telemetry samples, weather snapshots, tracked vehicles, and risk events recorded during one driving session. The entities and their main attributes are summarized in Table 4.1.

Table 4.1: Entities of the Logical Data Model

Entity	**Description and main attributes**
Trip	One driving session; start and end time, and the input mode (device sensors or CARLA simulation).
TelemetrySample	GPS-based reading; timestamp, speed, bearing, latitude, longitude, and validity flag.
WeatherSnapshot	Weather fetched for the current location; timestamp, condition, rain, and visibility.
DetectedObject	Tracked vehicle or obstacle; class, bounding box, estimated proximity, and closing status.
RiskEvent	Assessed risk; timestamp, risk score, level, contributing factors, and the Why and Advice messages.

Chapter 5

Implementation

The ContextDrive system has been implemented as an integrated advanced driver-assistance platform combining edge-based computer vision, GPS telemetry, environmental intelligence, and deterministic risk assessment. The implementation connects these diverse modules so that real-time driving data can be processed on-device and presented to the driver through a unified, smartphone-based interface.

5.1 Tools and Technologies

Frontend Development

The frontend of the system is developed using the Flutter framework with the Dart programming language. Flutter provides a highly responsive, cross-platform UI toolkit that enables the creation of a smooth, real-time camera dashboard. It handles the display of live video feeds, floating UI overlays for risk scores, and the presentation of transparent safety explanations.

Edge AI and Computer Vision

The core perception capabilities are powered by Google LiteRT (formerly TensorFlow Lite), which allows the YOLOv8 Nano model to execute efficiently directly on the smartphone hardware. This edge-AI approach ensures that video frames are processed locally in real-time, eliminating network latency and preserving driver privacy since video data never leaves the device.

Telemetry and Location Services

The system utilizes native smartphone GPS sensors accessed via Flutter location and geolocator packages. This provides the continuous stream of speed, heading, and coordinate data necessary for the risk engine to accurately monitor driving dynamics.

Environmental Data Services

The OpenWeatherMap API is used as the external cloud service for retrieving hyper-local weather data. Standard HTTP request libraries in Dart handle the API communication, fetching relevant environmental context based on the current GPS coordinates to enhance the system's situational awareness.

Table 5.1 summarizes how the tools are used in the different modules of the system.

Table 5.1: Technologies Used in the ContextDrive Modules

Module	Technology
Vision and object detection	YOLOv8 Nano model executed with Google LiteRT (formerly TensorFlow Lite) on the device.
Telemetry and GPS	Smartphone GPS accessed through the Flutter location and geolocator packages.
Environmental context	OpenWeatherMap API called with Dart HTTP libraries using the current GPS coordinates.
Risk engine and explainability	Deterministic rule-based engine implemented in Dart.
User interface	Flutter dashboard with live camera feed, bounding boxes, and risk overlays.
Simulation testing	CARLA simulator streaming virtual camera frames and telemetry to the phone over USB using ADB port forwarding.

5.2 Modules

Data Acquisition and Pre-processing Module

The data acquisition module collects the dynamic driving parameters required by the system. The implemented setup uses the smartphone's built-in sensors to obtain real-time video frames from the rear camera and speed, location, and bearing data from the GPS receiver. Weather information is obtained through an external weather service to provide environmental context such as visibility and precipitation. The collected values are validated and synchronized before being passed to the processing modules, ensuring that the input data is robust for risk evaluation.

Vision and Object Detection Module

The vision module is responsible for identifying potential hazards on the road ahead. The implemented model uses a lightweight YOLOv8 Nano architecture optimized for mobile devices. It processes the live camera frames to detect vehicles and obstacles, generating

bounding boxes and estimating proximity. The continuous object detection results are used by the risk assessment module to determine safe following distances and potential collision risks.

Environmental Context Module

The environmental context module provides critical situational awareness based on the vehicle's current location. The system queries the OpenWeatherMap API to retrieve real-time weather conditions. By considering factors like fog, heavy rain, or low visibility, this module provides contextual weights that dynamically adjust the sensitivity of the risk assessment algorithms, converting raw weather data into actionable driving intelligence.

Risk Assessment Module

The risk assessment module evaluates the integrated driving data to compute a continuous driving risk score. The system employs a deterministic, rule-based engine that compares vehicle speed, object proximity, and weather conditions against predefined safety thresholds. Instead of merely logging data, the module classifies the current driving situation into distinct risk levels (e.g., Safe, Warning, Critical) to facilitate immediate driver response.

Transparent Explainability Module

To make the automated risk assessments easier to understand and trust, an explainability module generates explicit reasoning for its decisions. Whenever a warning or critical alert is triggered, the system analyzes the contributing factors (e.g., "High speed in low visibility" or "Following too closely"). This approach ensures that the driver understands exactly why an alert was issued, reducing cognitive overload and improving the interpretability of the assistance system.

User Interface and Dashboard Module

The user interface module provides a seamless and intuitive dashboard for the driver. The mobile application overlays the live camera feed with real-time bounding boxes, the current risk score, safety alerts, and transparent recommendations. It is designed to deliver critical information efficiently without distracting the driver from the road.

5.3 Algorithms

5.3.1 Methods and Techniques

The ContextDrive system integrates various methods and techniques to support drivers in making informed, safe decisions. The system combines computer vision-based object detection, GPS telemetry tracking, weather data analysis, deterministic risk assessment, and transparent explainability mechanisms. These techniques work together to provide

personalized and data-driven safety recommendations.

Computer Vision-Based Object Detection
The system uses the YOLOv8 Nano model via Google LiteRT to detect vehicles and obstacles in the live camera feed. This lightweight model processes camera frames efficiently on-device, providing bounding boxes and proximity estimations for critical road elements.

GPS Telemetry Tracking
The system collects real-time speed, location, and bearing data from the smartphone's built-in GPS receiver. This telemetry data is used to continuously monitor the vehicle's dynamic state and movement patterns.

Weather Data Analysis
Weather and visibility information is obtained using the OpenWeatherMap API based on current GPS coordinates. Parameters such as visibility distance, precipitation, and extreme weather alerts are incorporated to adjust the risk sensitivity of the system.

Deterministic Risk Assessment
The system employs a rule-based deterministic risk engine to calculate a continuous driving risk score. It evaluates multiple factors, including vehicle speed, proximity to detected objects, and adverse weather conditions, to classify the driving situation into distinct risk levels.

Transparent Explainability
To build trust and prevent cognitive overload, the system generates clear, explicit explanations alongside its warnings. This ensures the driver understands exactly which environmental or behavioral factors contributed to the current safety alert.

Real-Time Driver Interface
The application provides a responsive Flutter-based dashboard that overlays the risk score, safety alerts, and actionable recommendations directly onto the live camera feed, ensuring seamless integration into the driving experience.

5.3.2 Temporal Object Tracking and Approach Detection

Detections from consecutive frames are associated into tracks, and the growth of the bounding-box area over time is used as a cue that a vehicle is approaching. A track must be observed repeatedly before it can indicate closing movement, so that an isolated noisy detection does not trigger a warning. A developing approach can be flagged before the configured near-proximity threshold is reached. Algorithm 1 gives the procedure.

Algorithm 1 Temporal tracking and approach detection

Require: Detections D_t at timestamp t, active tracks T, matching threshold τ, minimum observations N_{min}, growth threshold g_{th}, near threshold A_{near}
Ensure: Updated tracks with proximity and closing status

for all detection $d \in D_t$ **do**
    match d to the track in T with highest overlap
    if overlap $\geq \tau$ **then**
        append (area of d, t) to the history of that track
    else
        create a new track from d
    end if
end for
for all track $k \in T$ **do**
    if k not seen for the allowed time **then**
        remove k
        continue
    end if
    if observations of $k < N_{min}$ **then**
        mark k as not closing; **continue**
    end if
    $g \leftarrow$ rate of change of the area of k over the recent window
    k.closing $\leftarrow (g > g_{th})$
    if area of $k \geq A_{near}$ **then**
        k.proximity $\leftarrow$ NEAR
    else if k.closing **then**
        k.proximity $\leftarrow$ DEVELOPING APPROACH
    else
        k.proximity $\leftarrow$ FAR
    end if
end for

5.3.3 Context-Aware Risk Assessment

The risk engine is a deterministic, rule-based procedure. It combines the speed from the GPS, the proximity and closing status of the tracked vehicles, the weather, and the time of day. Adverse weather and night driving increase the sensitivity of the rules. When the required hazards occur together, such as a near or closing vehicle combined with high speed or adverse conditions, the situation is classified as critical. If an input is unavailable or stale, the engine records a data limitation instead of assuming favourable conditions. Algorithm 2 gives the procedure.

Algorithm 2 Context-aware risk assessment

Require: Driving context C = (speed v, tracks T, weather w, time of day, data quality)
Ensure: Risk result (score, level, factors, Why, Advice)

$F \leftarrow \emptyset$
if any input of C is missing or stale **then**
    add DATA LIMITATION to F ▷ never treated as favourable
end if
$k \leftarrow$ sensitivity factor from w and time of day ▷ rain, low visibility, night
if $v >$ speed threshold scaled by k **then**
    add SPEEDING to F
end if
if deceleration $>$ hard-braking threshold **then**
    add HARD BRAKING to F
end if
if acceleration $>$ rapid-acceleration threshold **then**
    add RAPID ACCELERATION to F
end if
if a track in T has proximity NEAR **then**
    add NEAR VEHICLE to F
end if
if a track in T is closing **then**
    add CLOSING VEHICLE to F
end if
if w is adverse **then**
    add ADVERSE WEATHER to F
end if
score $\leftarrow$ sum of the weights of the factors in F
if required compound-hazard conditions hold **then**
    level $\leftarrow$ CRITICAL
else if score $\geq$ warning threshold **then**
    level $\leftarrow$ WARNING (CAUTION)
else
    level $\leftarrow$ SAFE (MONITORING)
end if
Why $\leftarrow$ short description of the most significant factors in F
Advice $\leftarrow$ recommended action for level and factors
return (score, level, F, Why, Advice)

5.3.4 Alert Confirmation and Voice Prioritisation

To avoid false alarms and repeated announcements, each event type has its own confirmation delay, and voice alerts are rate-limited. A higher-severity event is allowed to interrupt the rate limit. Algorithm 3 gives the procedure.

Algorithm 3 Alert confirmation and voice prioritisation

Require: Risk result r with event type e and level s; confirmation delay d_e; cooldown c

Ensure: Decision to display and announce the alert

display the risk card for r
if s is SAFE **then**
    return
end if
if event e has not persisted for at least d_e **then**
    return
end if
if $s >$ severity of the last announcement **then**
    announce r by voice ▷ escalation is always allowed
else if time since the last announcement of $e \geq c$ **then**
    announce r by voice
else
    suppress the voice alert ▷ duplicate announcement
end if

5.4 Key Code Snippets

The listings below show representative code for two of the modules: reading speed and bearing from the GPS, and retrieving the weather from the OpenWeatherMap API. They illustrate how the Flutter location packages and the Dart HTTP libraries are used for the telemetry and environmental context modules.

```
final stream = Geolocator.getPositionStream(
  locationSettings: const LocationSettings(
    accuracy: LocationAccuracy.best,
    distanceFilter: 0,
  ),
);

stream.listen((Position p) {
  final speedKmh = p.speed * 3.6;   // m/s to km/h
  final bearing = p.heading;
  telemetry.update(speedKmh, bearing, p.latitude, p.longitude);
});
```

Listing 5.1: Subscribing to speed and bearing updates using the geolocator package

```
Future<WeatherInfo?> fetchWeather(double lat, double lon) async {
  final uri = Uri.https('api.openweathermap.org', '/data/2.5/weather', {
    'lat': '$lat', 'lon': '$lon', 'units': 'metric', 'appid': apiKey,
  });
  try {
    final res = await http.get(uri).timeout(const Duration(seconds: 5));
    if (res.statusCode != 200) return null;   // treated as unavailable data
    return WeatherInfo.fromJson(jsonDecode(res.body));
  } catch (_) {
    return null;                              // never assume clear weather
  }
}
```

Listing 5.2: Fetching the current weather for the GPS coordinates

Chapter 6

Testing

6.1 Test Strategy

A multi-level testing strategy was used to evaluate the functionality and reliability of ContextDrive. The application integrates on-device YOLO vehicle detection, temporal object tracking, GPS filtering, environmental context, CARLA simulation, and a deterministic risk assessment engine.

The use of the CARLA simulator for validating driving functions follows approaches reported in recent work on simulation-based validation [6, 7]. The testing strategy comprises the following levels:

- **Unit Testing:** Validates individual components, including camera-frame protocol parsing, telemetry processing, bounding-box tracking, GPS filtering, risk classification, and alert confirmation policies.
- **Integration Testing:** Verifies communication between the CARLA simulator, the Flutter application, the virtual camera stream, and the telemetry service.
- **System Testing:** Examines application behaviour using simulated driving inputs, including vehicle approach, motion events, and changing environmental conditions.
- **Interface Testing:** Verifies the display of camera frames, vehicle detections, telemetry values, contextual indicators, and risk assessments on the mobile application.

6.2 Test Cases

The following test cases cover the principal functional modules of ContextDrive.

1. **TC-01 (Camera Stream Protocol):** Process packets containing camera-frame metadata and image data.

Expected Result: Valid frames are decoded, fragmented packets are handled correctly, and duplicate or out-of-order frames are rejected.

Status: **Pass.**

2. **TC-02 (Telemetry Processing and Mode Isolation):** Process CARLA telemetry containing vehicle speed, scenario information, and simulation timestamps while physical GPS updates are present.

   Expected Result: Valid telemetry is parsed correctly, and CARLA mode uses simulator inputs without interference from physical GPS updates.

   Status: **Pass.**

3. **TC-03 (GPS Filtering):** Evaluate stationary GPS drift and sudden speed outliers.

   Expected Result: Stationary noise and transient outliers are filtered according to the implemented rules.

   Status: **Pass.**

4. **TC-04 (Temporal Object Tracking and Early Approach Detection):** Track vehicles across consecutive frames with gradual or rapid bounding-box expansion, including timestamped sequences representing an approaching vehicle.

   Expected Result: Vehicle tracks are maintained during plausible approaches, isolated noisy detections do not automatically indicate closing movement, and a developing approach can be classified before the configured near-proximity threshold is reached.

   Status: **Pass.**

5. **TC-05 (Risk Classification and Compound Hazard Assessment):** Evaluate hard braking, rapid acceleration, vehicle proximity, speeding, and adverse environmental conditions using controlled inputs.

   Expected Result: The risk engine applies the configured rules, selects the appropriate severity and explanation, and assigns CRITICAL severity when the required compound-hazard conditions are satisfied.

   Status: **Pass.**

6. **TC-06 (Alert Confirmation and Voice Prioritisation):** Evaluate event-specific confirmation policies and repeated voice alerts followed by a higher-severity event.

   Expected Result: Configured confirmation delays are applied, duplicate announcements are rate-limited, and higher-severity escalation is permitted.

   Status: **Pass.**

7. **TC-07 (Data Quality and Stale Input Handling):** Evaluate unavailable environmental inputs and stale telemetry or visual data.

   Expected Result: The application represents data limitations appropriately and does not automatically treat missing inputs as favourable conditions.

   Status: **Pass.**

8. **TC-08 (Mobile Interface):** Examine risk cards, contextual indicators, and portrait and landscape layouts.

   Expected Result: The interface displays the relevant information without layout overflow in the tested cases.

   Status: **Pass.**

9. **TC-09 (CARLA Camera and Telemetry Integration):** Stream virtual-camera frames from CARLA to the Nothing Phone (2a) over USB and receive changing vehicle speed and environmental context.

   Expected Result: The application displays the incoming camera feed, YOLO vehicle bounding boxes, and updated telemetry values.

   Status: **Pass.**

6.3 Results of Unit, Integration, and System Testing

6.3.1 Unit Testing Outcomes

The Flutter test suite was executed, and the recorded execution output reported that **42 tests passed with zero reported failures**.

The tests covered camera-stream handling, telemetry parsing, GPS filtering, object tracking, risk classification, voice-alert prioritisation, and user-interface behaviour.

The object-tracking tests included developing vehicle approaches, rapid bounding-box expansion, stationary objects, receding vehicles, and noisy detections. The risk-engine tests also covered distinct braking and acceleration classifications, compound hazard conditions, environmental advisories, and data-quality handling.

6.3.2 Integration Testing Outcomes

Integration testing was carried out using the CARLA simulator and the Nothing Phone (2a).

The following functionality was observed:

- The mobile device was recognised through Android Debug Bridge (ADB).
- CARLA virtual-camera frames were transmitted over USB using ADB port forwarding on port 8088.
- The ContextDrive application displayed the incoming camera feed and YOLO vehicle bounding boxes.
- CARLA vehicle speed was received and updated on the mobile application through the existing telemetry connection.
- Environmental context from the CARLA scenario was reflected in the application.

These observations demonstrate the basic integration of the CARLA simulation, video transport, mobile perception pipeline, and telemetry interface.

6.3.3 System Testing Outcomes

ContextDrive was exercised with the CARLA manual driving rig to verify the integrated camera and telemetry workflow.

During operation, the simulated vehicle could be driven while camera frames were streamed to the mobile application. YOLO detections were displayed as bounding boxes, and the application received changing vehicle speed and environmental context.

The automated risk-engine tests additionally verified the configured responses to approaching vehicles, distinct motion events, and combinations of visual and environmental conditions. These results establish functional verification of the tested software rules and basic operation of the integrated mobile pipeline.

Chapter 7

Results and Discussion

This chapter presents the results obtained with ContextDrive. It shows the application interface, analyzes the performance observed during testing, compares the system with the existing work reviewed in Chapter 2, and discusses how well the objectives were met.

7.1 Screenshots

Figure 7.1 and Figure 7.3 show the ContextDrive dashboard in two driving situations. In both screens the left panel displays the weather and day or night indicators, the voice alert toggle, the current speed, the number of detected vehicles, the risk focus, and the GPS status. The right panel shows the live camera feed with bounding boxes around the detected vehicles and the risk card.

In Figure 7.1 the system is in the low-risk MONITORING state. The risk card is green and explains that normal driving conditions are present, and the advice is to continue driving safely. In Figure 7.3 a vehicle is near and the system raises a CAUTION alert. The amber risk card states why the alert was issued, shows the observed condition and the supporting evidence from the camera view, and gives the advice to check the road ahead and to increase the following distance if needed.

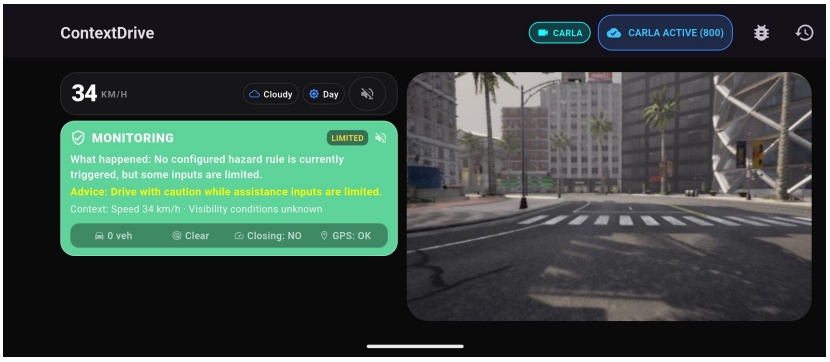


Figure 7.1: ContextDrive - Low risk event

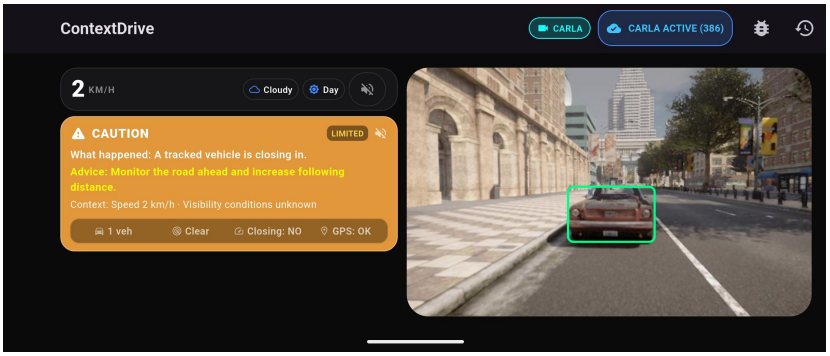


Figure 7.2: ContextDrive - High risk event

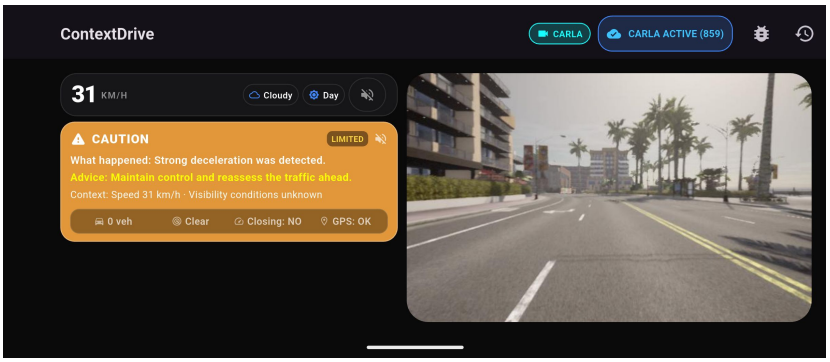


Figure 7.3: ContextDrive - High risk event

7.2 Performance Analysis

The performance of ContextDrive was evaluated through the multi-level testing described in Chapter 6. Table 7.1 summarizes the evidence for each aspect of the system.

Table 7.1: Summary of the Evaluation Results

Aspect	Evidence	Outcome
Automated tests	Flutter test suite covering camera-stream handling, telemetry parsing, GPS filtering, object tracking, risk classification, voice-alert prioritisation, and the user interface.	42 passed, 0 failed
Functional test cases	Test cases TC-01 to TC-09 covering the camera protocol, telemetry, GPS filtering, tracking, risk rules, alerts, data quality, interface, and CARLA integration.	All passed
Object tracking	Developing approaches, rapid bounding-box expansion, stationary objects, receding vehicles, and noisy detections.	Behaved as configured
Risk engine	Braking and acceleration events, compound hazards, environmental advisories, and data-quality handling.	Behaved as configured
Integration	CARLA virtual-camera frames sent to a Nothing Phone (2a) over USB using ADB port forwarding, with live speed and environmental context.	Camera feed, detections, and telemetry displayed
Interface	Risk cards, contextual indicators, and portrait and landscape layouts.	No layout overflow in tested cases

The results show that the tracking rules distinguish a developing approach from isolated noisy detections, that the risk engine escalates to CRITICAL only when the required compound-hazard conditions are met, and that missing or stale inputs are reported as data limitations. The voice alert policy rate-limits duplicate announcements while still allowing a higher-severity event to be announced. In the CARLA integration test, the application received the virtual camera stream and the changing vehicle speed and environmental context, and displayed the YOLO detections as bounding boxes while the simulated vehicle was driven with the manual driving rig.

These results establish the functional verification of the tested software rules and the basic operation of the integrated mobile pipeline. Measurements such as frame rate, inference latency, battery use, and thermal behaviour on the device, and trials on real roads, are not part of the present evaluation and are listed as future enhancements.

7.3 Comparison with Existing Systems

Table 7.2 compares ContextDrive with the systems reviewed in Chapter 2 on the main capabilities of a context-aware driver assistance system.

Table 7.2: Comparison of ContextDrive with Existing Systems

System	Camera perception	GPS telemetry	Weather and time context	Explainable output	Smartphone based
Muhammad et al. [1]	No	Yes	Yes	Yes	Yes
Aulia et al. [2]	Yes	No	No	No	Partial
Masello et al. [4]	No	Yes	Yes	Yes	No
Nguyen et al. [3]	Yes	No	No	No	No
ContextDrive	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

The comparison is based on the descriptions of the systems in Chapter 2. The work of Peintner et al. [5] studies how explanations should be presented rather than a sensing system, so it is not included in the table, but its findings guided the design of the short Why and Advice messages. The telemetry-based systems do not use the camera and so cannot see the road scene, while the camera-based systems do not use the weather and time context to adjust their sensitivity. The explainable context models require heavy computation and are suited to post-trip analysis, and the embedded implementation is tied to a specific hardware platform. ContextDrive brings the camera, GPS, weather, and time context together in a single on-device application on an ordinary smartphone, and explains each warning with a short rationale.

7.4 Discussion

The expected outcome of the project was a user-friendly, smartphone-based driver assistance system that supports drivers in making informed decisions using real-time camera, GPS, and environmental information, and that explains the factors behind each warning. Table 7.3 relates the objectives set in Chapter 1 to the evidence obtained.

Table 7.3: Objectives and the Evidence Obtained

Objective	**Evidence**
Develop a smartphone-based context-aware system for real-time risk assessment	Integrated on-device pipeline of detection, tracking, GPS, weather, risk engine, and dashboard; CARLA integration test (TC-09).
Integrate camera, GPS, weather, and time information	Driving context built from all four sources; telemetry, GPS filtering, and data-quality tests (TC-02, TC-03, TC-07).
Generate a risk score and timely, transparent recommendations	Risk classification and compound-hazard rules (TC-05), alert confirmation and voice prioritisation (TC-06), and the Why and Advice cards in Figures 7.1 and 7.3.
Improve driver awareness without additional hardware	Runs on a standard smartphone with its rear camera and GPS; interface tested in portrait and landscape layouts (TC-08).

The vision module identifies leading and nearby vehicles on the device, and the risk module evaluates speed, proximity, and adverse weather together instead of treating each hazard separately. The explainability module identifies the critical factors, for example a near and closing vehicle, that triggered a warning. This keeps the explanation concise, as recommended by the literature, so that the driver is not overloaded. Because all processing is carried out on the phone, the video never leaves the device, which supports privacy and a fast response. Overall, the results indicate that a low-cost smartphone can provide integrated, explainable, context-aware driving assistance. The limits of the present evaluation are discussed in the next chapter.

Chapter 8

Conclusion and Future Scope

8.1 Summary

The ContextDrive project provides an integrated advanced driver-assistance platform that combines edge-based computer vision, GPS telemetry tracking, environmental intelligence, deterministic risk assessment, and transparent explainability. By using continuous vehicle dynamics and environmental data, the system helps drivers identify potential road hazards and receive real-time, actionable safety guidance. The integration of explainable AI improves the transparency of risk assessments, while the edge-processing architecture supports privacy and rapid response times. The mobile-first approach further provides accessible safety features without requiring expensive hardware modifications. Overall, the project aims to support drivers in making informed, data-driven decisions and adopting safer, more proactive driving habits.

8.2 Limitations

The following limitations of the present work should be noted.

- The evaluation consists of automated software tests and an integration test with the CARLA simulator on one phone. Trials on real roads and in a variety of traffic conditions have not yet been carried out.
- The risk engine is rule-based with predefined thresholds. As reported for other threshold-based systems, such values may need recalibration for different regions and driving cultures [1].
- A single monocular camera provides no direct depth measurement, so the proximity of vehicles is estimated from the detections and is only approximate.

- The accuracy of the detector depends on the visibility of the scene. Conditions that are poorly represented in the training data, such as heavy fog or poorly lit night driving, may reduce its performance [2].
- Weather information requires an internet connection. Without it, the system can only report the missing input as a data limitation.
- Reading explanations on a phone screen can distract a driver who is actively controlling the vehicle [5].
- Frame rate, inference latency, battery consumption, and thermal behaviour on the device were not measured quantitatively.

8.3 Future Enhancements

- Conduct field trials on real roads with different drivers, vehicles, and traffic conditions, and measure the accuracy of the warnings.
- Benchmark the frame rate, latency, battery use, and thermal behaviour on a range of smartphones, and explore model quantization and hardware acceleration to improve efficiency.
- Deliver warnings through distinct audio tones, haptic feedback, and peripheral color changes in addition to text, to reduce visual distraction [5].
- Calibrate and personalize the thresholds for different regions and drivers using recorded trips.
- Improve the robustness of the detector at night and in rain or fog by fine-tuning on data that represents these conditions [2].
- Extend the perception to more hazards, such as pedestrians, cyclists, lane markings, and traffic signs, and to estimating distance more accurately.
- Add weather forecasts, trip history, and post-trip summaries to help drivers review and improve their driving habits.
- Extend the application to additional platforms and use richer simulated scenarios for repeatable testing.

References

[1] A. S. Muhammad, L. Chen, and C. Wang, "Personalised driver risk assessment with adaptive feedback using crowdsensed telemetric data," *IET Intelligent Transport Systems*, vol. 19, no. 1, p. e70071, 2025.

[2] U. Aulia, I. Hasanuddin, M. Dirhamsyah, and N. Nasaruddin, "A new cnn-based object detection system for autonomous mobile robots based on real-world vehicle datasets," *Heliyon*, vol. 10, no. 15, 2024.

[3] H.-H. Nguyen, D. N.-N. Tran, L. H. Pham, and J. W. Jeon, "Optimizing monocular driving assistance for real-time processing on jetson agx xavier," *IEEE Access*, vol. 12, pp. 71 853–71 865, 2024.

[4] L. Masello, G. Castignani, B. Sheehan, M. Guillen, and F. Murphy, "Using contextual data to predict risky driving events: A novel methodology from explainable artificial intelligence," *Accident Analysis & Prevention*, vol. 184, p. 106997, 2023.

[5] J. Peintner, C. Manger, M. Berger, and A. Riener, "Balancing transparency and control: The impact of ai explanation detail on user perception in automated vehicles," *Transportation Research Interdisciplinary Perspectives*, vol. 34, p. 101742, 2025.

[6] C. Gómez-Huélamo, J. Del Egido, L. M. Bergasa, R. Barea, E. López-Guillén, F. Arango, J. Araluce, and J. López, "Train here, drive there: ROS based end-to-end autonomous-driving pipeline validation in CARLA simulator using the NHTSA typology," *Multimedia Tools and Applications*, vol. 81, pp. 4213–4240, 2022.

[7] S. M. Cardozo and V. Hlaváč, "CARLA hardware-in-the-loop simulation environment for autonomous driving," *SAE International Journal of Connected and Automated Vehicles*, vol. 9, no. 2, pp. 289–309, 2026.

[8] A. Afdhal, K. Saddami, M. Arief, S. Sugiarto, Z. Fuadi, and N. Nasaruddin, "Mxt-yolov7t: An efficient real-time object detection for autonomous driving in mixed traffic environments," *IEEE Access*, vol. 12, pp. 178 566–178 585, 2024.

[9] S.-Y. Park and J. Lee, "Tdo-slam: Traffic sign and dynamic object based visual slam," *IEEE Access*, vol. 12, pp. 24 569–24 582, 2024.

[10] A. Jarndal, H. Tawfik, A. I. Siam, I. Alsyouf, and A. Cheaitou, "A real-time vision transformers-based system for enhanced driver drowsiness detection and vehicle safety," *IEEE Access*, vol. 13, pp. 1790–1803, 2025.

[11] B. B. Gupta, A. Gaurav, K. T. Chui, and V. Arya, "Deep learning model for driver behavior detection in cyber-physical system-based intelligent transport systems," *IEEE Access*, vol. 12, pp. 62 268–62 278, 2024.

[12] Y. Kim and J. Shin, "Robust object detection against multi-type corruption without catastrophic forgetting during adversarial training under harsh autonomous-driving environments," *IEEE Access*, vol. 11, pp. 26 862–26 876, 2023.

[13] N. M. Alahdal, F. Abukhodair, L. H. Meftah, and A. Cherif, "Real-time object detection in autonomous vehicles with yolo," in *Procedia Computer Science*, vol. 246. Elsevier, 2024, pp. 2792–2801.

[14] S. K. Dhatrika, D. R. Reddy, and N. K. Reddy, "Real-time object recognition for advanced driver-assistance systems (adas) using deep learning on edge devices," in *Procedia Computer Science*, vol. 252. Elsevier, 2025, pp. 25–42.

[15] X. Han, J. Chang, and K. Wang, "Real-time object detection based on yolo-v2 for tiny vehicle object," in *Procedia Computer Science*, vol. 183. Elsevier, 2021, pp. 61–72.